

TP 3

TranspireDesFesses

Patrice DOUSSOT

1 FEATURE 5

Le programme va proposer des grilles faites au hasard.

Au lancement du programme, il demandera à l'utilisateur la difficulté voulue, celle correspondra au nombre de cases vides.

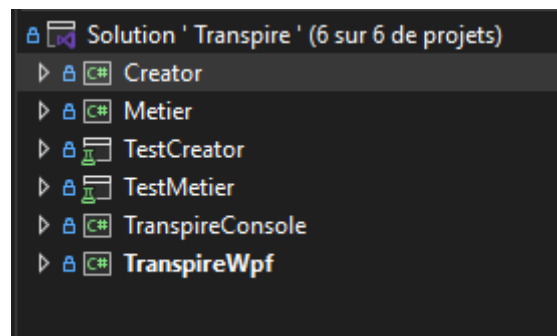
Cette difficulté sera comprise entre 1 et 6 (compris). Le nombre de case vide sera 10 fois la difficulté. Le programme s'arrêtera sur son plus grand de cases vides s'il ne trouve pas de solutions.

En effet à partir de la difficulté 6, il est dur de trouver au hasard une grille avec 60 cases vides.

1.1 PROJET CREATOR

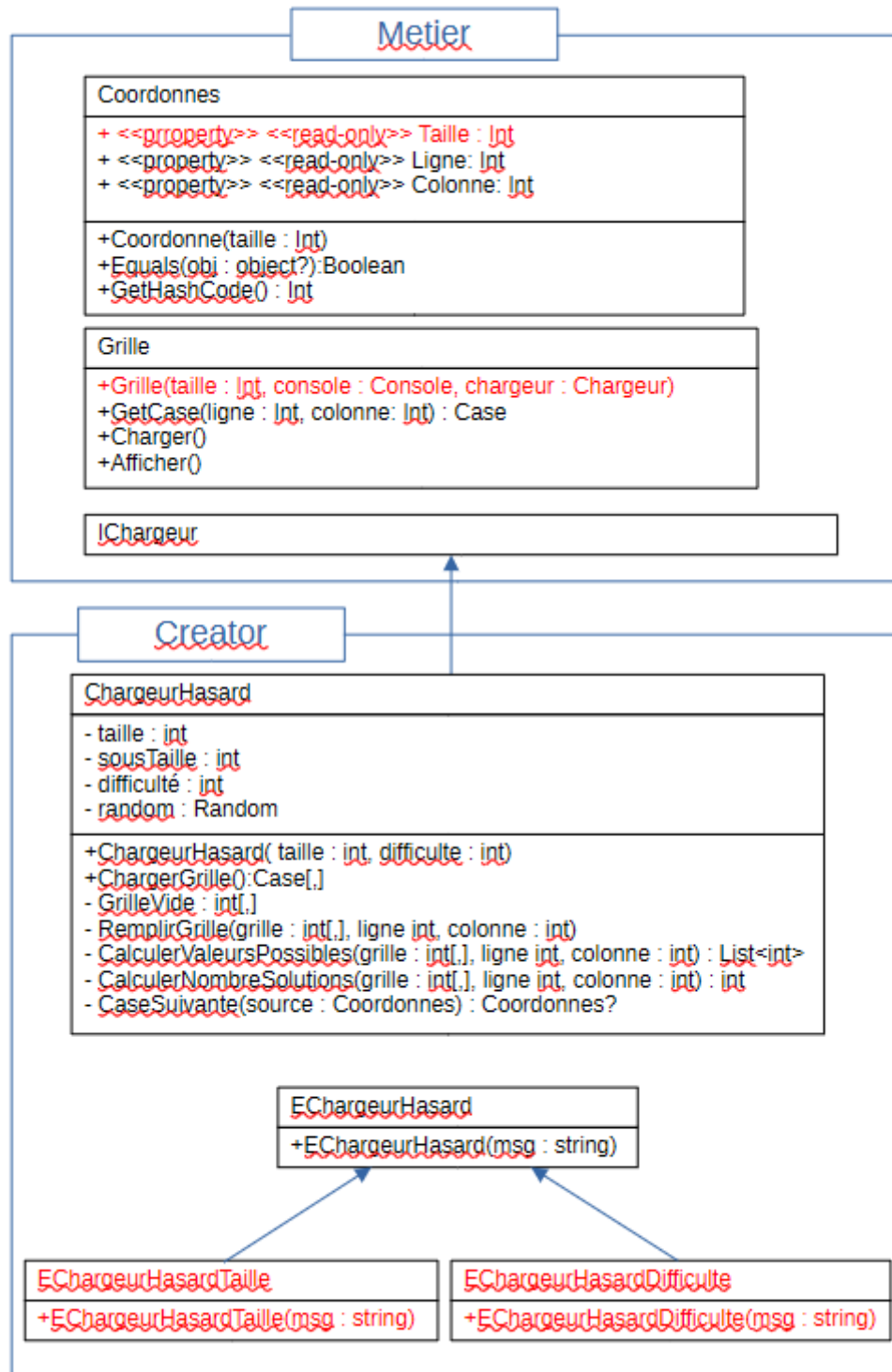
Commencer par créer un nouveau projet, de type Bibliothèque de classes appelé Creator et son projet de test Test Creator.

Voici la tête de votre solution :



1.2 DIAGRAMME UML

Comme d'habitude, les lignes ajoutées ou modifiées sont en rouges



1.3 RÈGLES METIER

Coordonnees.Taille devient une propriété.

Le constructeur de Grille déclenche une exception EGrilleTaille si la taille n' est pas un carré (seul 4,9 et 16 sont acceptées) .

Le constructeur de ChargeurHasard déclenche :

- EchargeurHasardTaille si la taille indiquée n' est pas un carré
- EchargeurHasardDifficulte si la difficulté n'est pas comprise entre 1 et 6 (compris)

GrilleVide remplit une grille de 0.

RemplirGrille remplit la grille a partir de la position indiquée.

CalculerValeurPossibles donne toutes les valeurs restantes pour la position indiquée.

CalculerNombreSolutions calcule le nombre de solutions possibles a partir de la position indiquée.

CaseSuivante donne la case qui suit la case indiquée, retourne null si c'est la dernière case de la grille.

1.4 ALGORITHMES

Voici l'algorithme de la fonction ChargerGrille :

Créer une grille vide

Stocker la liste des cases restantes

encorePossible = vrai

faire

 Stocker la liste des cases restantes pour ce tour

 boucleCase = vrai

 faire

 choisir une case au hasard

 enlever cette case

 si le nombre de solutions possibles avec cette case en moins = 1

 La case est bonne, donc boucleCase = faux

 Augmenter le nombre de cases supprimées

 on enlève cette case de la liste des cases restantes

 sinon

 on remet cette case

 on enlève cette case de la liste des cases restantes pour ce tour

 Si cette liste est vide

 encorePossible = faux

 boucleCase = faux

 tant (boucleCase = vrai)

tant que (Nombre de cases à supprimer < difficulté x 10) et (encorePossible = vrai)

On convertit le tableau d'entier en tableau de Case

Pour simplifier cet algorithme, on ne fait pas de backTracing (remonter d' un niveau).

A vous de trouver les autres algorithmes.

Ne faite pas d' algorithmes à boucle infini, pensez au manque de chance.

La récursivité peut vous aider.

Ne fait pas de test pour la classe ChargeurHasard.

2 FEATURE 6

On demande la difficulté pour la version graphique au lancement du programme.

C' est facile.